`postcss.config.mjs`	Pinceau automatique CSS : transforme le CSS pour assurer la compatibilité multi-navigateurs (souvent avec Tailwind CSS).
`eslint.config.mjs`	Correcteur de code : surveille la syntaxe et les erreurs de style en temps réel dans VS Code.

> **Conseil :** Dans cet atelier Pipeline, le fichier le plus important est le Jenkinsfile : c'est lui qui définit le voyage de votre code de votre ordinateur jusqu'au site final en production.

4. Déploiement du projet sur Heroku

Cette section explique comment déployer votre application Next.js sur la plateforme cloud Heroku.

4.1 Préparer le projet localement

Avant de lancer les commandes Heroku, assurez-vous d'être dans le dossier de votre projet dans le terminal.

4.2 Séquence de déploiement

Étape 1 — Connexion à Heroku

```
heroku login
```

Une fenêtre s'ouvrira dans votre navigateur pour vous authentifier via votre compte Heroku.

Étape 2 — Création de l'application

```
heroku create nom-de-votre-application
```

> **Important :** Si le nom est déjà utilisé, Heroku vous demande d'en choisir un autre ou en génère un automatiquement.

Étape 3 — Déploiement du code

```
git push heroku main
# ou si votre branche principale s'appelle encore 'master' :
git push heroku master
```

Étape 4 — Ouverture du site

```
heroku open
```

Votre navigateur s'ouvre automatiquement sur l'URL de votre application déployée.

4.3 Fichiers essentiels pour Heroku

Pour que le déploiement fonctionne, vérifiez la présence de ces deux éléments à la racine du projet :

package.json — Scripts obligatoires

```
{
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start"
  }
}
```

Procfile — Démarrage Heroku

```
web: npm start
```

> ⚠ **Important :** Ce fichier s'appelle exactement 'Procfile', sans extension, avec un P majuscule.

5. Configuration Docker & Jenkins

Installation et gestion du serveur d'automatisation CI/CD dans un conteneur Docker isolé.

5.1 Pré-requis

- Lancer Docker Desktop sur la machine hôte avant toute chose.
- Vérifier que Docker fonctionne avec la commande : docker ps

5.2 Installation de Jenkins via Docker

Lancez la commande complète suivante pour télécharger et démarrer Jenkins :

```
docker run -d \
  --name jenkins \
  -p 8080:8080 \
  jenkins/jenkins:lts
```

Cette commande permet de lancer **Jenkins** (un outil d'automatisation et d'intégration continue) à l'intérieur d'un conteneur **Docker**

Explication de chaque paramètre

Commande	Description
`docker run`	Télécharge et démarre le conteneur Jenkins
`-d`	Mode arrière-plan (détaché) — ne bloque pas le terminal
`--name jenkins`	Donne un nom au conteneur pour le retrouver facilement
`-p 8080:8080`	Mappage de ports 8080 : → port interne Jenkins 8080
`jenkins/jenkins:lts`	Image officielle Jenkins en version LTS (Long Term Support)

Accès à l'interface Jenkins :

```
http://localhost:8080
```

5.3 Gestion du conteneur

Commande	Description
`docker ps -a`	Inventaire de tous les conteneurs (actifs et stoppés)
`docker start jenkins`	Redémarre Jenkins après un redémarrage de la machine
`docker rm jenkins`	Supprime le conteneur pour repartir de zéro en cas d'erreur

5.4 Récupérer la clé de sécurité initiale

Lors du premier démarrage, Jenkins est verrouillé. Récupérez le mot de passe initial avec :

```
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Copiez ce code et collez-le sur la page web de Jenkins lors de la première connexion.

5.5 Résumé de l'installation

Étapes d'installation Jenkins
1. Lancez Docker Desktop
2. Exécutez la commande docker run complète dans votre terminal
3. Attendez environ 1 minute le démarrage complet
4. Allez sur http://localhost:8080 dans votre navigateur
5. Utilisez docker exec pour obtenir votre code secret
6. Collez le code sur la page web de Jenkins et finalisez l'installation

6. Identifiants et Automatisation Pipeline

Configuration finale de Jenkins pour mettre en place le pipeline CI/CD complet.

6.1 Accès Jenkins

```
http://localhost:8080/
```

Champ	Valeur
User	_____
Password	_____

6.2 Plugins requis

- Installer le plugin NodeJS dans Jenkins.
- Ce plugin permet à Jenkins d'exécuter npm install, npm run build, etc.

- Installation via : Gérer Jenkins → Gérer les plugins → Disponibles → NodeJS Plugin

6.3 Logique du pipeline

Le pipeline CI/CD automatise le flux complet depuis chaque modification de code :

Flux du Pipeline Automatisé
Étape 1 — Source : Récupération du code depuis le dépôt Git
Étape 2 — Test : Vérification automatique de la qualité du code
Étape 3 — Build : Compilation de l'application Next.js (npm run build)
Étape 4 — Deploy : Envoi automatique sur Heroku via git push heroku

6.4 Débogage

> **Important :** Pour tout problème, lisez attentivement le message d'erreur dans la console Jenkins. Comprendre le message est la clé d'une bonne résolution de problème.

7. Bilan

À l'issue de cet atelier, vous avez mis en place une chaîne complète de déploiement continu.

Ce que vous avez réalisé

Récapitulatif des compétences acquises
✅ Créer et configurer un projet Next.js from scratch
✅ Versionner votre code avec Git (init, add, commit, branch)
✅ Comprendre la structure et le rôle de chaque fichier de configuration
✅ Déployer une application sur Heroku via ligne de commande
✅ Installer et configurer Jenkins dans un conteneur Docker
✅ Automatiser un pipeline CI/CD de bout en bout

> **Conseil :** La maîtrise du Pipeline as Code est une compétence clé en entreprise. Ces pratiques sont utilisées quotidiennement dans les équipes DevOps modernes.